AN IOT-ENABLED INTELLIGENT CLASSROOM MONITORING FRAMEWORK FOR REAL-TIME OCCUPANCY AND NOISE DETECTION

BHARATH SASIKUMAR – 24619249

PRATHAMESH CHOHALE – 24619239

JONATHAN WHEELER – 24619214

VELS INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)

MINI PROJECT REPORT

Submitted to the

SCHOOL OF ENGINEERING

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

in Partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY - CSE

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI – 600 117

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report "AN IOT-ENABLED INTELLIGENT CLASSROOM MONITORING FRAMEWORK FOR REAL-TIME OCCUPANCY AND NOISE DETECTION", is the Bonafide work of BHARATH SASIKUMAR – (24619249), PRATHAMESH CHOHALE – (24619239), JONATHAN WHEELER – (24619214) who carried out the project under my supervision. Certified further, that to the best of my knowledge the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD
Dr. K. Ulaga Priya M.E., (Ph.D)
Associate Professor & Head
Department of AI&ML
School of Engineering
VISTAS
Chennai

SIGNATURE OF THE SUPERVISOR
Mrs. SM. SOWMIYA M.E., (Ph.D)
Assistant Professor
Department of AI&ML
School of Engineering
VISTAS
Chennai

Submitted for the Project Work and Viva - Voce examination held on…… at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support we received from the positive minds around us in making this venture a successful one. The infrastructural support with all kinds of lab facilities has been a motivating factor in the completion of this project work, all because of the FOUNDER & CHAIRMAN, Shri. Dr. ISHARI K. GANESH, and it is with great pleasure that we take this opportunity to thank him. We are grateful to our HEAD OF THE DEPARTMENT, Dr. K. Ulaga Priya, for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day. We extend our warmest thanks to our INTERNAL GUIDE, Mrs. SM. Sowmiya, Department of AI&ML, who has patiently guided us and provided invaluable advice and help. We would like to express our greatest gratitude to our staff members, Librarian, and non-teaching staff members of the Computer Centre, who have helped and supported us throughout. Last but not the least, we wish to thank our parents and our friends for their individual support and interest, who have inspired and encouraged us to go our own way; without them we would be unable to complete this project.

BHARATH SASIKUMAR – 24619249

PRATHAMESH CHOHALE – 24619239

JONATHAN WHEELER – 24619214

CHAPTER NO.	TABLE OF CONTENT	PAGE NO.
	ABSTRACT	i
	LIST OF FIGURES	ii
	LIST OF ABBREVIATIONS	iii
1	**INTRODUCTION**	1
	1.1 Overview of the project	1
	1.2 Scope and objective	3
2	**LITERATURE REVIEW**	5
	2.1 Literature overview	5
	2.2 Literature survey	5
3	**SYSTEM DESIGN**	9
	3.1 Existing system	9
	3.2 Proposed system	9
	3.3 System architecture	10
	3.4 Hardware requirements	11
	3.5 Functional and non-functional requirements	13
4	**IMPLEMENTATION**	14
	4.1 Methodology	14
	4.2 Implementation	15
	4.2.1 Sensor acquisition and pre-processing module	16
	4.2.2 Camera occupancy module	16
	4.2.3 Radar and fusion module	17
	4.2.4 Noise detection module	17
	4.2.5 Classification and alert module	18
	4.2.6 Dashboard module	18
	4.2.7 Overall algorithm	18
	4.2.8 Design considerations	19
	4.3 Testing and validation plan	19
5	**CONCLUSION AND FUTURE WORK**	20
	5.1 Results	20
	5.1.1 Example monitoring cycle	20
	5.1.2 Dashboard output	21
	5.1.3 Expected impact	21
	5.2 Conclusion	22
	5.3 Future work	22
	Sample source code	23
	5.4 Sample input and output	26
	REFERENCES	28

ABSTRACT

Classrooms are managed by timetable and manual observation, so overcrowding, unused seats and excessive noise often go unnoticed until a complaint is raised. This project presents an IoT-enabled intelligent classroom monitoring framework for real-time occupancy and noise detection. The system is built around a single ESP32 development board that reads an OV2640 camera, an LD2410 24 GHz radar sensor and a KY-038 / MAX4466 sound sensor. The camera counts students and identifies free seats, while the radar provides backup presence detection in poor light or when a camera frame is unusable. Sound readings are processed and smoothed to classify the noise level as Quiet, Normal or Excessive. Occupancy, expressed as the ratio of the student count to the room capacity, and noise are combined into a single classroom status that is shown on a real-time dashboard, with alerts for sustained excessive noise or over-capacity conditions. The framework is privacy-friendly: images are processed on the device and only counts and status are transmitted. The system is a low-cost prototype under development, and the example results in this report are simulated for illustration.

Keywords: IoT, ESP32, classroom monitoring, occupancy detection, noise detection, mmWave radar, sensor fusion, smart classroom.

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO.
Figure 3.1	System architecture of the proposed classroom monitoring node	10
Figure 4.1	Methodology pipeline (ESP32-driven)	14
Figure 4.2	Interconnection of the sensors with the ESP32	16
Figure 4.3	Camera and radar fusion decision logic	17
Figure 4.4	Occupancy classes as a function of the occupancy ratio	18
Figure 5.1	Simulated example of one monitoring cycle (illustrative, not measured)	20
Figure 5.2	Dashboard output of the Smart Classroom Monitor	21
Figure 5.3	Sample input: hardware setup of the prototype	26
Figure 5.4	Sample output: Arduino IDE with ESP32 Dev Module	26

LIST OF ABBREVIATIONS

Abbreviation	Full Form
IoT	Internet of Things
ESP32	Espressif 32-bit microcontroller with Wi-Fi
mmWave	Millimetre Wave
PIR	Passive Infrared
CNN	Convolutional Neural Network
ADC	Analog-to-Digital Converter
UART	Universal Asynchronous Receiver-Transmitter
SCCB	Serial Camera Control Bus
ROI	Region of Interest
JSON	JavaScript Object Notation
HTTP	Hypertext Transfer Protocol
API	Application Programming Interface
Wi-Fi	Wireless Fidelity
dB	Decibel
ρ	Occupancy ratio (student count / capacity)
GPIO	General Purpose Input/Output

CHAPTER 1. INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

1.1.1 Introduction

A classroom is one of the most heavily used spaces in any educational institution, yet it is still managed largely by timetable and manual observation. A room is allotted to a class for a fixed period and nobody checks, while the period is running, how many students are actually present, how many seats are unused, or how loud the room has become. As a result, overcrowded rooms, half-empty rooms and disruptive noise usually go unnoticed until a teacher complains or an administrator inspects the room by hand.

The Internet of Things (IoT) offers a practical way to close this gap. Low-cost microcontrollers with built-in Wi-Fi, small camera modules, millimetre-wave (mmWave) radar sensors and analog sound sensors are now cheap enough to be installed in every room of a campus. When these devices are combined on a single board, they can report the state of a classroom continuously, without any manual effort from the staff.

This project presents an IoT-enabled intelligent classroom monitoring framework for real-time occupancy and noise detection. The framework is built around a single ESP32 development board to which three sensors are attached: an OV2640 camera that counts students and identifies free seats, an LD2410 24 GHz radar that provides a backup presence signal when the camera cannot see clearly, and a KY-038 / MAX4466 sound sensor that measures the noise level. The ESP32 fuses these inputs, classifies the room into easily understood states, and shows the result on a dashboard with alerts for sustained excessive noise or over-capacity conditions. The system is a prototype under development; the example values shown in this report are simulated and are marked as such wherever they appear.

1.1.2 Problem statement

In most institutions, classroom usage is monitored by manual observation. This approach has several shortcomings:

- **Overcrowding:** there is no automatic student count, so a room can exceed its safe seating capacity without anyone being aware of it.

- **Inefficient seating:** free desks are not identified, and spare capacity in one room cannot be used to relieve pressure in another.
- **Excessive noise:** disruptive noise from the room or its surroundings goes undetected and is addressed only after it has affected teaching.
- **No real-time awareness:** problems are noticed only after the fact, usually through a complaint or an inspection round, so no timely action is possible.

The core problem, therefore, is the absence of a low-cost, always-on system that can tell administrators and teachers how many people are in a room, whether the room is empty or full, and whether the noise level is acceptable, at the moment the information is needed.

1.1.3 Motivation

Existing research and products address individual parts of this problem. Some systems mark attendance from face images; others detect the presence of a teacher; others measure noise and raise an alert. Very few combine occupancy and noise on one inexpensive board, and fewer still make the two readings interact, for example by judging whether a given noise level is reasonable for a room that is nearly empty or completely full. The motivation for this work is to demonstrate that a single ESP32-based node, costing a small fraction of a commercial occupancy-analytics product, can provide a combined and more informative view of a classroom.

A second motivation is privacy. A camera inside a classroom raises legitimate concerns. The proposed framework therefore processes images on the device itself and transmits only derived values such as the student count, the radar state, the noise class and a timestamp. No image leaves the room.

1.1.4 Overview of the proposed approach

The proposed approach can be summarised as a five-stage pipeline: sensor data acquisition, data collection, data processing, condition classification and user output. Each stage is implemented on the ESP32 and described in detail in Chapter 4. The key idea is sensor fusion: the camera is the primary source for counting, the radar is a secondary source that keeps the room correctly marked as occupied in dim light or when a camera frame is unusable, and the

sound sensor provides a continuous noise reading. The three readings are reduced to two labels, an occupancy state and a noise state, which are combined into one classroom status.

1.2 SCOPE AND OBJECTIVE

1.2.1 Objectives of the project

The main objectives of the project are:

- To build a compact IoT node for real-time occupancy and noise detection in a classroom.
- To count students and identify free desks using a camera module.
- To use a radar sensor as backup presence detection in poor lighting or when the camera frame is unusable.
- To combine occupancy and noise readings into a single classroom status.
- To present the real-time status to administrators and teachers on a dashboard, with alerts for sustained excessive noise and over-capacity conditions.

1.2.2 Scope of the project

The framework is intended as an assistive monitoring tool for administrators and teachers rather than a replacement for them. Its scope includes the following:

- **Occupancy monitoring:** estimating the number of students present and the number of free seats, expressed as an occupancy ratio against the room capacity.
- **Presence detection:** confirming whether anyone is in the room using a 24 GHz radar even when the camera cannot provide a usable image.
- **Noise monitoring:** measuring the sound level, smoothing it, and classifying it as Quiet, Normal or Excessive.
- **Status reporting:** showing a timestamped classroom status on a dashboard and raising an alert on sustained excessive noise or over-capacity.
- **Applicability:** lecture rooms, tutorial rooms, laboratories, seminar halls and similar indoor teaching spaces on a campus.

1.2.3 Limitations

The system in its present form has the following limitations. It is a prototype under development, and the figures presented in the results section are simulated for illustration and are not measurements taken from the prototype. Camera-based counting is affected by occlusion and seating layout. The radar sensor confirms presence but does not give an accurate head-count. The noise sensor is an inexpensive module and, unless calibrated against a reference sound-level meter, its readings are approximate. Finally, a single node covers a single room; scaling to a whole campus is listed as future work.

1.2.4 Organisation of the report

Chapter 1 introduces the problem, objectives and scope. Chapter 2 reviews the existing literature and identifies the research gap. Chapter 3 describes the existing and proposed systems, the system architecture and the hardware and software requirements. Chapter 4 explains the methodology and the implementation of each module. Chapter 5 presents the conclusion, future work, the core source code and the illustrative results, followed by the references.

CHAPTER 2. LITERATURE REVIEW

2.1 LITERATURE OVERVIEW

Research related to smart classrooms spans several areas: automated attendance marking, human presence detection, low-cost surveillance, environmental noise monitoring and integrated smart-classroom platforms. The common thread is the use of inexpensive sensing hardware, usually built around ESP32-class microcontrollers, to replace tasks that were traditionally done by hand.

Attendance systems use cameras and face-recognition networks to identify students as they enter or sit in a room. Presence-detection systems use passive infrared (PIR) sensors, radar or audio to decide whether a person is in a room, which is useful for energy control and for faculty tracking. Noise-monitoring systems measure the sound level and notify an operator when a threshold is crossed. Integrated platforms attempt to combine attendance, energy management and audio analysis, but often at the price of high hardware cost and complexity.

The following section surveys five representative works. For each work, the problem it addresses, its core concept and its limitations are noted. These limitations are the starting point for the design decisions of the proposed framework.

2.2 LITERATURE SURVEY

2.2.1 ESP32-CAM and CNN face-recognition attendance

This work (ICCCES 2026) addresses attendance marking in classrooms. An ESP32-CAM module captures images of students and a convolutional neural network (CNN) performs face recognition so that attendance is recorded automatically and in real time. The approach removes the need for roll-calls and demonstrates that low-cost hardware can run vision tasks in an educational setting.

Its main limitation is that recognition accuracy degrades in poor lighting, which is common in classrooms with dim corners or when projectors are in use. It also provides no seating information and no noise analytics, so the output is limited to a list of names.

2.2.2 Multi-modal IoT and AI faculty presence detection

This work (IJRSI 2025) addresses the detection of faculty presence. It fuses a PIR sensor, face recognition and voice detection so that the system can decide whether a faculty member is present in a cabin or classroom. The use of several modalities makes the decision more robust than any single sensor, which is a principle that the present project also follows.

The limitation is its narrow target: it concerns faculty only and does not count students or describe how full a room is.

2.2.3 ESP32 Wi-Fi CAM surveillance robot

This work (IATMSI 2025) describes a mobile surveillance robot that carries an ESP32 Wi-Fi camera and streams video for visual monitoring. It shows how little hardware is required to obtain a remote view of a space.

The robot must physically move to cover an area and it performs no occupancy counting, so it cannot give a continuous, quantitative picture of a classroom. Streaming video also raises privacy concerns that the proposed framework avoids by keeping images on the device.

2.2.4 Real-time noise detection and alert system

This work (ResearchGate 2025) addresses noise detection and alerting. Sound levels are collected and sent to a FastAPI back-end that stores them in an InfluxDB time-series database and raises alerts when readings exceed a limit. The architecture is scalable and well suited to dashboards and history.

Its limitations are that the thresholds are fixed and that noise is not linked to occupancy. The same sound level means something very different in an empty room (possible disturbance from outside) and in a full room (normal class activity), and a fixed threshold cannot make that distinction.

2.2.5 AI and IoT-enabled smart classroom monitoring

This work (IJET 2025) proposes a smart-classroom platform combining attendance, energy control and audio summaries. It is the broadest of the surveyed systems and shows the value of integrating several functions.

However, it is complex and hardware-heavy, which makes it expensive to deploy in every room of an institution and difficult to maintain.

2.2.6 Summary of the surveyed work

Table 2.1 Summary of the literature survey

S.No	Paper title (venue)	Problem concerned	Concept	Limitations
1	ESP32-CAM & CNN Face Recognition (ICCCES 2026)	Attendance marking in classrooms	Real-time face-recognition attendance	Degrades in poor light; no seating or noise analytics
2	Multi-Modal IoT & AI Faculty Presence Detection (IJRSI 2025)	Faculty presence detection	PIR + face + voice fusion	Faculty only; no student counting
3	ESP32 Wi-Fi CAM Surveillance Robot (IATMSI 2025)	Visual surveillance	Low-cost visual monitoring	Robot must move; no occupancy counting
4	Real-Time Noise Detection & Alert System (ResearchGate 2025)	Noise detection and alerts	FastAPI + InfluxDB alerts	Fixed thresholds; not linked to occupancy
5	AI & IoT-Enabled Smart Classroom Monitoring (IJET 2025)	Smart classroom monitoring	Attendance, energy control and audio summaries	Complex and hardware-heavy

2.2.7 Research gap

The survey shows that each existing work covers one aspect of classroom monitoring: attendance, presence or noise. The integrated platform covers several aspects but at the cost of complexity. The gaps that remain open, and that this project targets, are:

- No surveyed low-cost system combines student counting, backup presence detection and noise sensing on a single board.
- Camera-only systems fail in poor light, and no surveyed design uses a second sensor to back the camera up.
- Noise alerts use fixed thresholds and are not interpreted in the context of how many people are in the room.
- Privacy is rarely addressed explicitly; many designs stream or store images.

The contribution of this project is a single ESP32 node with camera and radar fusion and a noise-aware status, in which images stay on the device and only counts and status are transmitted.

CHAPTER 3. SYSTEM DESIGN

3.1 EXISTING SYSTEM

In the existing arrangement, classroom monitoring is manual. Room allocation follows a timetable, and the actual usage of a room is known only to the people inside it. An administrator who wants to know whether a room is full must visit it; a teacher who is disturbed by noise from a neighbouring room must walk over and ask. Where technology is used, it is usually restricted to one function, for example a biometric attendance terminal at the door or a CCTV camera that records but does not interpret.

The existing system therefore suffers from delayed detection, lack of quantitative data, dependence on human attention and, in the case of CCTV, a privacy burden without a corresponding analytical benefit. No historical data on occupancy or noise is available for planning, so decisions about room allocation are made on assumption rather than evidence.

3.1.1 Disadvantages of the existing system

- Occupancy and noise are not measured; they are only observed informally.
- Issues are discovered after the fact.
- No data for timetable and room planning.
- Single-purpose tools do not give a combined picture of the classroom.
- Camera recordings, where present, are not analysed and raise privacy concerns.

3.2 PROPOSED SYSTEM

The proposed system replaces manual observation with a compact, low-cost IoT node. One ESP32 development board reads a camera, a radar sensor and a sound sensor, processes the readings locally and sends a small status message over Wi-Fi to a dashboard. The design principles are as follows.

- **Low cost:** all parts are inexpensive, widely available hobby-grade modules.
- **Sensor fusion:** the camera is the primary source for counting; the radar is trusted when the camera frame is unusable or reports zero while the radar sees presence.
- **Context-aware status:** occupancy and noise are combined into one classroom status rather than reported separately.

- **Privacy by design:** images are processed on the device; only counts, states and timestamps are transmitted.
- **Real-time feedback:** the dashboard updates continuously and alerts are raised on sustained excessive noise or over-capacity.

3.2.1 Advantages of the proposed system

- Gives administrators automatic, continuous information on every monitored room.
- Identifies free seats and spare capacity.
- Keeps the room marked as occupied in dim light through radar backup.
- Reduces false alarms by requiring noise to be sustained before alerting.
- Provides timestamped data that can be used for room planning.

3.3 SYSTEM ARCHITECTURE

The architecture of the proposed system, shown in Fig. 3.1, has three layers: the input or sensing layer, the processing layer and the classification or output layer.

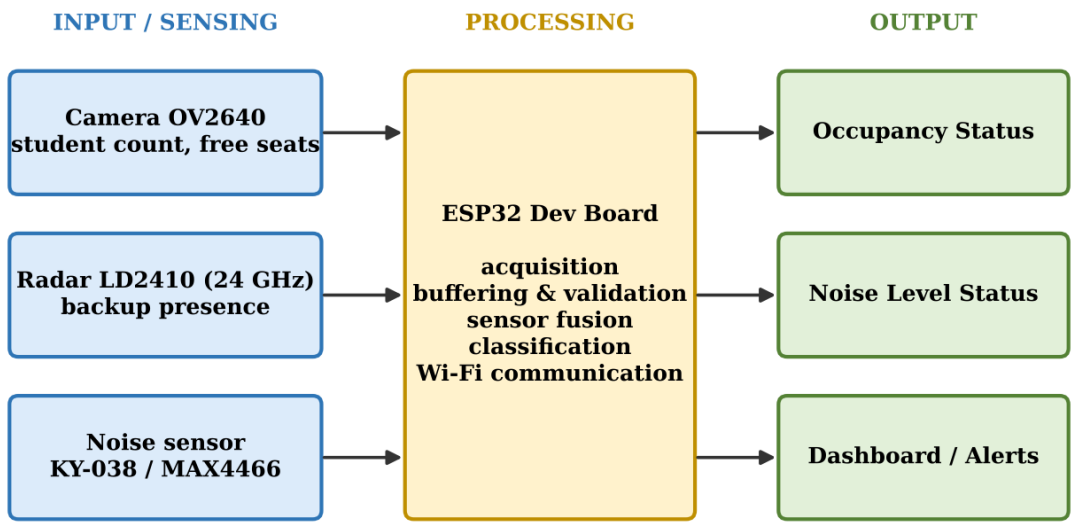


Fig. 3.1 System architecture of the proposed classroom monitoring node

3.3.1 Input / sensing layer

The sensing layer consists of three sensors. The OV2640 camera captures frames of the classroom, which are used to count students and to decide which seats are occupied. The

LD2410 24 GHz radar detects the presence of people from their micro-movements and works independently of lighting. The KY-038 or MAX4466 sound sensor produces an analog signal proportional to the ambient sound level.

3.3.2 Processing layer

The ESP32 development board forms the processing layer. It acquires data from all three sensors, buffers it, validates it (discarding bad frames and out-of-range radar values and smoothing the sound readings), applies the fusion rule, classifies the room state and communicates the result wirelessly over Wi-Fi.

3.3.3 Classification / output layer

The output layer produces three results: a classroom occupancy status, a noise-level status and a user-facing output consisting of a dashboard tile and an alert indication. Occupancy is expressed as Empty, Partly occupied, Near capacity or Over capacity. Noise is expressed as Quiet, Normal or Excessive.

3.4 HARDWARE REQUIREMENTS

The hardware used in the prototype is listed in Table 3.1 and described below.

Table 3.1 Hardware requirements

Component	Function in the system	Key specification
ESP32 development board	Processing, buffering, fusion and Wi-Fi communication	Dual-core, built-in Wi-Fi and Bluetooth
OV2640 camera module	Student counting and free-desk detection	2 MP image sensor, SCCB control
LD2410 radar sensor	Backup presence detection in poor light	24 GHz mmWave, UART output
KY-038 / MAX4466 sound sensor	Sound-level monitoring	Analog output to ADC
Breadboard and jumper wires	Circuit assembly	Prototype wiring
USB cable and stable 5 V supply	Power and programming	5 V DC

3.4.1 ESP32 development board

The ESP32 is a low-cost system-on-chip with a dual-core processor, integrated Wi-Fi and Bluetooth, multiple UART and analog-to-digital converter channels, and a dedicated camera

interface. Its combination of processing power and connectivity at a very low price is what makes a single-board node feasible.

3.4.2 OV2640 camera module

The OV2640 is a 2-megapixel CMOS image sensor widely used with the ESP32. It delivers frames at resolutions from QQVGA up to UXGA and supports JPEG compression in hardware. In this project it provides the frames from which students are counted and seats are marked as occupied or free.

3.4.3 LD2410 radar sensor

The LD2410 is a 24 GHz millimetre-wave radar module that detects both moving and stationary human presence. Because it senses reflected radio waves rather than light, it keeps working in darkness, with a dim projector-lit room, or when the camera lens is partly obstructed. It reports its detection state to the ESP32 over a serial (UART) link.

3.4.4 Noise sensor

The KY-038 and MAX4466 are electret-microphone modules that output an analog voltage following the sound pressure at the microphone. The ESP32 samples this voltage with its ADC, measures the signal amplitude over a short window and converts it to an approximate decibel value that is smoothed before classification.

3.4.5 Connections and power

The sensors are connected to the ESP32 as listed in Table 3.2. The pin numbers are typical values used in the firmware listing in Section 5.5 and must be matched to the actual wiring of the board in use.

Table 3.2 Sensor connections to the ESP32

Sensor	Sensor pin	ESP32 connection
LD2410 radar	TX	GPIO16 (UART2 RX)
LD2410 radar	RX	GPIO17 (UART2 TX)
LD2410 radar	VCC / GND	5 V / GND
KY-038 / MAX4466	Analog out (AO)	GPIO34 (ADC input)
KY-038 / MAX4466	VCC / GND	3.3 V / GND
OV2640 camera	Data, clock and SCCB lines	Camera connector pins of the board

Sensor	Sensor pin	ESP32 connection
Whole node	Power	Stable 5 V via USB

The camera and the Wi-Fi radio together draw current peaks, so a stable 5 V supply capable of delivering enough current is required; a weak USB port or a long thin cable can cause the board to reset. The breadboard and jumper wires are used only for the prototype; a soldered board or a printed circuit board is recommended for a permanent installation.

3.5 FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

3.5.1 Functional requirements

- **Sensing:** the node shall read the camera, radar and sound sensor in every monitoring cycle.
- **Counting:** the node shall estimate the number of occupied and free seats from the camera frame.
- **Fusion:** the node shall use the radar presence flag whenever the camera result is unusable or zero while presence is detected.
- **Classification:** the node shall assign an occupancy class and a noise class to every cycle.
- **Reporting:** the node shall publish the latest status through an HTTP GET /data endpoint.
- **Alerting:** the dashboard shall indicate sustained excessive noise and over-capacity conditions.

3.5.2 Non-functional requirements

- **Low cost:** the complete node should use only inexpensive, easily available modules.
- **Privacy:** no image shall be stored or transmitted off the device.
- **Responsiveness:** the dashboard shall refresh about once per second.
- **Reliability:** invalid sensor readings shall be rejected without crashing the firmware.
- **Maintainability:** thresholds, capacity and calibration values shall be configurable constants.
- **Usability:** the status shall be understandable by non-technical staff without training.

CHAPTER 4. IMPLEMENTATION

4.1 METHODOLOGY

The methodology follows the pipeline shown in Fig. 4.1: sensor data acquisition, data collection, data processing, condition classification and user output. The pipeline runs continuously on the ESP32; each pass through it is called a monitoring cycle.

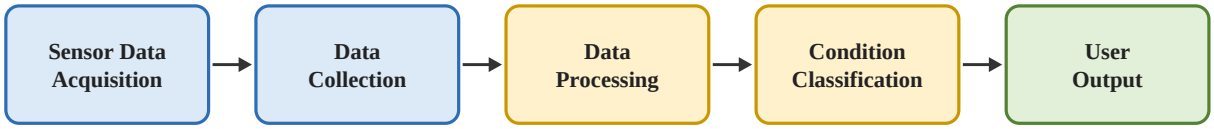


Fig. 4.1 Methodology pipeline (ESP32-driven)

4.1.1 Data acquisition

In each cycle the ESP32 captures one camera frame, reads the current presence state from the radar and samples the analog sound sensor over a short window. The three sensors are read at different natural rates; the firmware therefore stores the latest value from each sensor in a buffer so that a complete reading set is always available at the moment of fusion.

4.1.2 Data collection

Collected readings are placed in small buffers. The camera buffer holds the most recent frame and its derived count. The radar buffer holds the latest presence flag with its age. The sound buffer holds a short history of amplitude readings so that smoothing can be applied. Buffering decouples the sensor timing from the classification step and makes the system tolerant of occasional missed readings.

4.1.3 Data processing and validation

Raw sensor data is not used directly. Bad camera frames, for example frames that are too dark, saturated or corrupted, are discarded. Radar values that are out of range or stale are ignored. Sound readings are smoothed with a moving average so that a single loud event, such as a door slamming, does not by itself change the noise class.

4.1.4 Fusion rule

The camera is trusted as the primary source of the student count. If the camera frame is unusable, or if the camera reports zero people while the radar reports presence, the radar is

trusted and the room is marked as occupied, although the head-count is then unavailable. This rule prevents a dark or obstructed camera from making an occupied room look empty.

4.1.5 Classification

Occupancy is classified using the occupancy ratio, defined as the student count divided by the room capacity: ρ = count / capacity. The classes are Empty, Partly occupied, Near capacity and Over capacity (Table 4.1). Noise is classified as Quiet, Normal or Excessive from the smoothed sound level (Table 4.2). The threshold values are configurable parameters, initialised to the values below.

Table 4.1 Occupancy classification

Class	Condition on ρ	Meaning
Empty	$\rho = 0$ (and radar shows no presence)	Room is unused
Partly occupied	$0 < \rho < 0.8$	Spare seats available
Near capacity	$0.8 \leq \rho \leq 1.0$	Few seats remain
Over capacity	$\rho > 1.0$	More people than seats

Table 4.2 Noise classification

Class	Smoothed level	Action
Quiet	below 50 dB	None
Normal	50 dB to 70 dB	None
Excessive	above 70 dB	Alert if sustained

4.1.6 User output

The classification results are published by the ESP32 as a small JSON message through an HTTP GET /data endpoint. A web dashboard polls this endpoint once per second and shows the occupancy tile, the noise tile and any active alert. An alert is raised only when excessive noise is sustained for a defined period, or when the room is over capacity, which avoids reacting to momentary spikes.

4.2 IMPLEMENTATION

The firmware is organised into five modules that correspond to the stages above: sensor acquisition and pre-processing, camera occupancy module, radar and fusion module, noise

module and classification and alert module. The circuit connections between the ESP32 and the sensors are shown in Fig. 4.2.

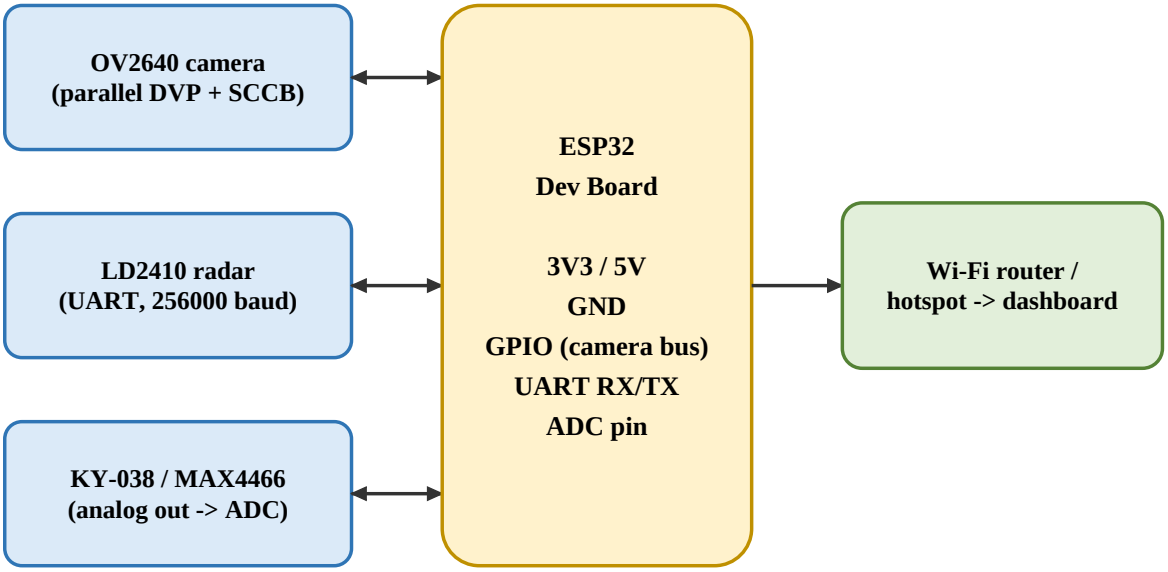


Fig. 4.2 Interconnection of the sensors with the ESP32

4.2.1 Sensor acquisition and pre-processing module

This module initialises the camera, the radar UART and the ADC channel at start-up and then reads them in each cycle. The camera is configured for a small frame size to keep memory use and processing time low. The radar is read through its library, which decodes the serial frames and exposes a presence flag. The sound pin is sampled many times in a short window.

Pre-processing rejects unusable data. A camera frame is rejected if the capture fails or if its mean brightness is outside an acceptable band. A radar reading is rejected if the sensor has not updated recently. These rejections feed the fusion rule: a rejected frame is the main reason for the radar backup to take over.

4.2.2 Camera occupancy module

The camera module converts a frame into two quantities: the number of students and the occupancy of each seat. The classroom layout is represented as a set of seat regions of interest (ROIs) defined once when the node is installed. Detection inside each region decides whether the seat is occupied, and the sum gives the student count. Free desks are the seats whose regions are not occupied.

Because everything is computed on the ESP32, the frame is discarded after the counts are produced. Only the numbers leave the module, which is the basis of the privacy-friendly design.

4.2.3 Radar and fusion module

The radar module returns a Boolean presence flag from the LD2410. The fusion module combines this flag with the camera result according to the decision logic in Fig. 4.3. If the camera frame is usable and the count is greater than zero, the camera count is used and the radar serves as confirmation. Otherwise the radar flag decides: if it shows presence the room is marked occupied with the count unavailable, and if it does not the room is marked empty.

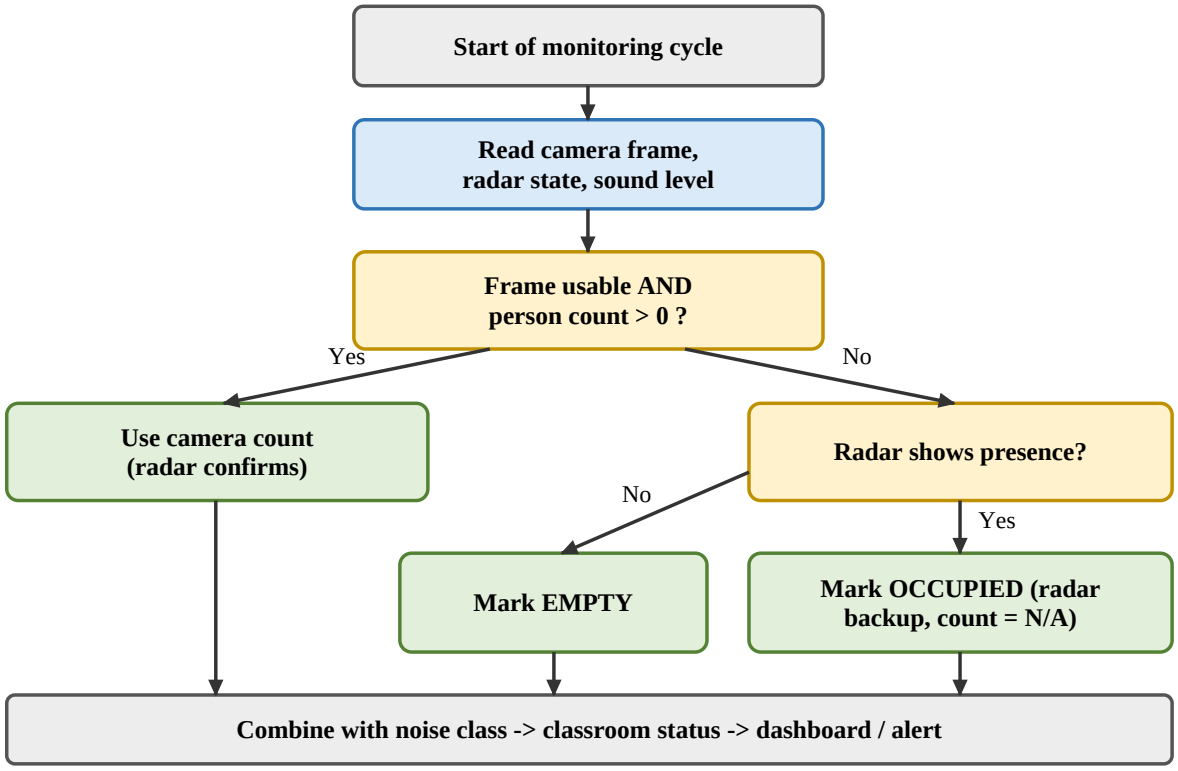


Fig. 4.3 Camera and radar fusion decision logic

4.2.4 Noise detection module

The noise module samples the analog pin over a short window, computes the peak-to-peak amplitude and converts it to an approximate decibel value using a calibration offset. The result is smoothed with a moving average. For calibration, the module is placed next to a reference sound-level meter and the offset is adjusted until the two agree across a range of sound levels. The smoothed value is then compared with the thresholds in Table 4.2.

To avoid false alerts, the module counts how long the level has stayed above the excessive threshold; an alert is generated only when this duration exceeds a configured limit.

4.2.5 Classification and alert module

This module computes ρ from the student count and the configured capacity and assigns the occupancy class shown in Fig. 4.4. It then combines the occupancy class and the noise class into the classroom status. The alert conditions are sustained excessive noise and over-capacity. The status, count, noise level and timestamp are then stored for the web server to publish.

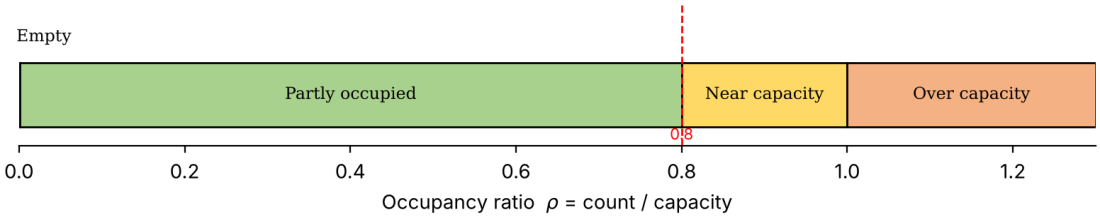


Fig. 4.4 Occupancy classes as a function of the occupancy ratio

4.2.6 Dashboard module

The dashboard is a single-page web application written in HTML, CSS and JavaScript. It runs on any laptop or phone connected to the same Wi-Fi network as the ESP32 and requests the /data endpoint every second. It displays the occupancy status, the noise level and the alert state, and it shows real sensor data only, with no random or placeholder values. The noise alert threshold is set to 70 dB.

4.2.7 Overall algorithm

The complete behaviour of the node in one monitoring cycle can be summarised by the following algorithm.

```
1.  Initialise camera, radar UART, ADC and Wi-Fi; load CAPACITY, thresholds,
calibration.
2.  Repeat every cycle:
3.     Capture frame; if capture fails or brightness is out of range, set frameOk =
false.
4.     If frameOk, count = number of occupied seat ROIs; else count = unavailable.
5.     Read radar presence flag; discard it if the reading is stale.
6.     If frameOk is false OR (count = 0 AND radar = presence):
7.         mark room OCCUPIED (radar backup), head-count = unavailable.
8.     Else if count = 0 AND radar = no presence: mark room EMPTY.
9.     Else compute rho = count / CAPACITY and assign the occupancy class.
10.    Sample the sound sensor, convert to dB, apply moving average.
11.    Assign the noise class; start or reset the "loud" timer.
12.    If loud timer > SUSTAIN or rho > 1.0: raise alert, else clear alert.
13.    Store count, classes, dB, alert and timestamp for the /data endpoint.
14.    Serve the latest values to the dashboard on request.
```

4.2.8 Design considerations

- **Memory:** the camera is set to a small frame size and one frame is held at a time so that the ESP32 does not run out of RAM.
- **Timing:** radar and sound readings are buffered so that a slow camera capture does not block the web server.
- **Robustness:** stale or out-of-range sensor values are ignored instead of being passed to the classifier.
- **Power:** a stable 5 V supply is recommended because the camera and Wi-Fi together cause current spikes that can reset the board.
- **Placement:** the camera should cover all seats, the radar should face the seating area and the microphone should be away from fans and the projector.

4.3 TESTING AND VALIDATION PLAN

Because the prototype is still under development, the testing approach is described here as a plan; the measured outcomes will be added when the tests are completed.

Table 4.3 Planned tests

Test	Method	Reference
Student count accuracy	Compare the camera + radar count with a manual count over several sessions	Manual headcount
Noise level accuracy	Compare the smoothed dB value with readings at the same position	Reference sound-level meter
Low-light behaviour	Dim the room and check that the status remains Occupied	Visual check and radar flag
Alert behaviour	Produce sustained loud sound and an over-capacity condition	Expected alert timing
Dashboard latency	Measure the time from sensing to display update	Timestamps

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 RESULTS

This section shows the output format of the system for one monitoring cycle. The values are a simulated example prepared for illustration; they were not measured from the prototype and must not be read as performance results.

5.1.1 Example monitoring cycle

Consider a classroom with a capacity of 40 seats. In the example cycle, the camera module reports 32 occupied seats and 8 free seats and the radar confirms presence. The occupancy ratio is ρ = 32 / 40 = 0.80, or 80 %, which places the room at the lower boundary of the Near-capacity class. The smoothed sound level falls between 50 dB and 70 dB, so the noise class is Normal and no alert is raised.

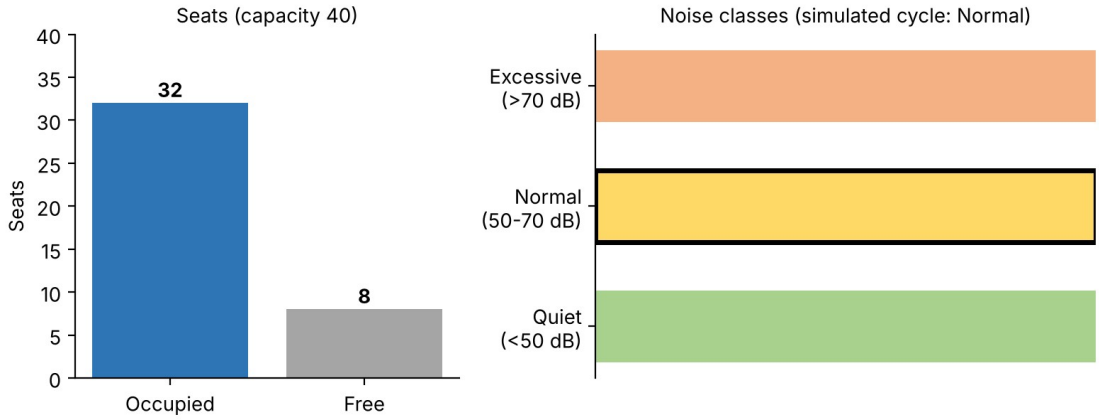


Fig. 5.1 Simulated example of one monitoring cycle (illustrative, not measured)

Table 5.1 Simulated dashboard output for the example cycle

Field	Value (simulated)
Room capacity	40 seats
Occupied seats	32
Free seats	8
Occupancy ratio ρ	0.80 (80 %)
Occupancy status	Near capacity
Noise class	Normal
Alert	None

5.1.2 Dashboard output

Fig. 5.2 shows the real-time dashboard of the prototype as seen in a web browser. The dashboard is hosted on Netlify and displays the latest readings sent by the ESP32 through the cloud backend. At the time of the capture the ESP32 status is CONNECTED and data are being received. The microphone tile shows a raw sound-sensor value of 367.0 with the status Normal, the radar tile shows CLEAR (no presence detected) and the camera tile shows ONLINE. The occupancy tile shows "--" with the message "Waiting for camera count", because no people count had been received from the camera program at that moment, and the alerts panel reports that there are no active alerts.

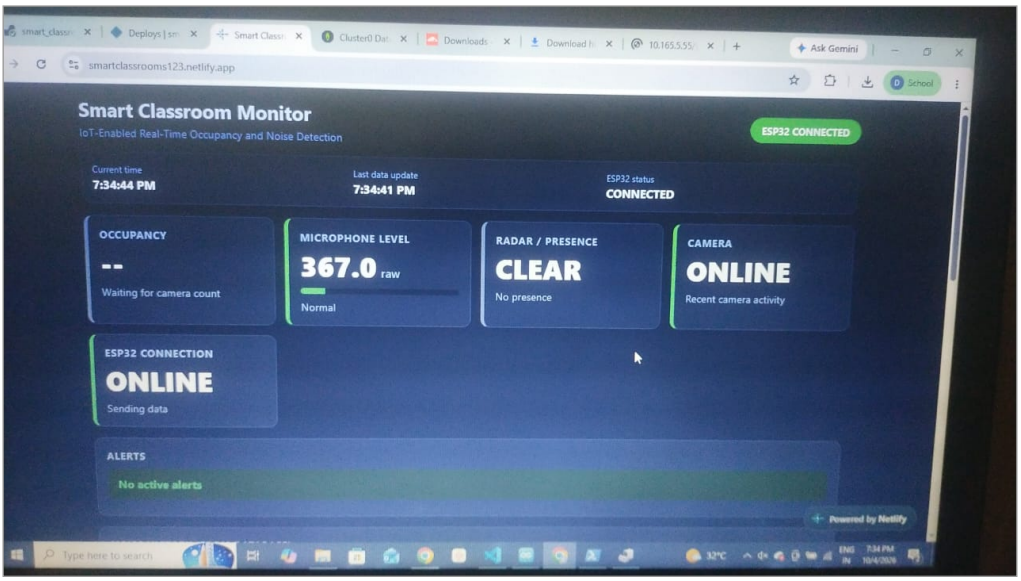


Fig. 5.2 Dashboard output of the Smart Classroom Monitor

5.1.3 Expected impact

When the planned tests are complete, the system is expected to offer the following benefits, which are stated as design goals and not as measured results:

- An accurate student count obtained from camera and radar fusion, checked against manual counts.
- Noise flagging checked against a reference sound-level meter.
- Low-light reliability, because the radar keeps the room marked as occupied when the camera is unreliable.
- Better decisions about room planning from timestamped occupancy and noise data.

5.2 CONCLUSION

This project presented a low-cost IoT framework that tells administrators and teachers what is happening in a classroom in real time. A single ESP32 node combines a camera, a 24 GHz radar and a sound sensor to determine how many students are present, which seats are free and how loud the room is, and it reduces these readings to an easily understood classroom status.

The literature survey showed that existing systems cover attendance, presence or noise in isolation, depend on fixed thresholds, degrade in poor lighting or are too complex and expensive. The proposed design addresses these gaps through camera and radar fusion, a noise-aware status and sustained-condition alerts. It is also privacy-friendly, because images remain on the device and only counts and status are transmitted.

The framework is a prototype under development. The example results in Section 5.1 are simulated for illustration; the accuracy of the counting, the calibration of the noise sensor and the low-light behaviour must be confirmed through the tests planned in Section 4.3 before any performance claims are made.

5.3 FUTURE WORK

- **Occupancy-based noise thresholds:** raise or lower the noise limit according to how many students are present, so that the same sound level is judged in context.
- **Timetable planning:** use the stored occupancy history to compare planned and actual room usage and to suggest better room allocation.
- **Campus-wide dashboard:** connect many nodes to a central server so that all rooms can be viewed together.
- **Improved counting:** evaluate lightweight neural models for person detection on the ESP32 to improve accuracy under occlusion.
- **Additional sensors:** temperature, humidity and air-quality sensors for a fuller picture of classroom comfort.
- **Cloud storage and analytics:** store long-term data in a database and provide reports and trend analysis.

SAMPLE SOURCE CODE

A simplified sample of the prototype code is listed below. The Wi-Fi name, backend URL and keys are placeholders that must be set for the installation.

A. ESP32 firmware (sample)

```
// ESP32 firmware (sample): reads noise + radar and posts them to the cloud
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>

const char* WIFI_SSID = "YOUR_WIFI_NAME";
const char* WIFI_PASS = "YOUR_WIFI_PASSWORD";
const char* API_URL   = "https://YOUR-BACKEND.onrender.com/api/data";
const char* API_KEY   = "YOUR_SECRET_KEY";

const int   NOISE_PIN = 34;           // KY-038 / MAX4466 analog output
const int   RADAR_PIN = 27;           // LD2410 OUT pin (HIGH = presence)
const float REF_P2P   = 20.0;         // calibration: reading in a quiet room
const float REF_DB    = 40.0;         // calibration: real dB of that room
const unsigned long SEND_MS = 3000;   // send to the cloud every 3 seconds

float noiseDb = 0;
unsigned long lastSend = 0;

float readNoiseDb() {
  unsigned long start = millis();
  int mx = 0, mn = 4095;
  while (millis() - start < 50) {             // 50 ms sampling window
    int s = analogRead(NOISE_PIN);
    if (s > mx) mx = s;
    if (s < mn) mn = s;
  }
  float p2p = max(1, mx - mn);
  float db = REF_DB + 20.0 * log10(p2p / REF_P2P);
  return constrain(db, 30, 120);
}

void sendToCloud() {
  if (WiFi.status() != WL_CONNECTED) return;
  WiFiClientSecure client;
  client.setInsecure();                       // demo only: skips certificate check
  HTTPClient http;
  if (!http.begin(client, API_URL)) return;
  http.addHeader("Content-Type", "application/json");
  http.addHeader("x-api-key", API_KEY);
  int radar = (digitalRead(RADAR_PIN) == HIGH) ? 1 : 0;
  String body = "{\"noise\":" + String(noiseDb, 1) +
                ",\"radar\":" + String(radar) + "}";
  int code = http.POST(body);
  Serial.printf("POST %s -> HTTP %d\n", body.c_str(), code);
  http.end();
}

void setup() {
  Serial.begin(115200);
  pinMode(RADAR_PIN, INPUT);
  analogReadResolution(12);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println(WiFi.localIP());
}

void loop() {
  noiseDb = 0.6 * noiseDb + 0.4 * readNoiseDb();   // moving-average smoothing
  if (millis() - lastSend >= SEND_MS) {
    lastSend = millis();
    sendToCloud();
  }
  delay(200);
}
```

SAMPLE SOURCE CODE

B. Backend API (sample)

```
// server.js (sample): Express + MongoDB API for the classroom monitor
require("dotenv").config();
const express  = require("express");
const cors     = require("cors");
const mongoose = require("mongoose");

const PORT       = process.env.PORT || 3000;
const API_KEY    = process.env.API_KEY;
const ESP_ONLINE = 10000;       // ESP32 is online if data arrived in the last 10 s
const CAM_ONLINE = 15000;       // camera is online if a count arrived in the last 15 s
const KEEP_SEC   = 60 * 60 * 24 * 7;   // old readings are deleted after 7 days

const Reading = mongoose.model("Reading", new mongoose.Schema({
  noise: { type: Number, required: true },
  radar: { type: Number, enum: [0, 1], required: true },
  createdAt: { type: Date, default: Date.now, expires: KEEP_SEC },
}));
const Occupancy = mongoose.model("Occupancy", new mongoose.Schema({
  count: { type: Number, required: true, min: 0, max: 500 },
  createdAt: { type: Date, default: Date.now, expires: KEEP_SEC },
}));

const app = express();
app.use(express.json({ limit: "10kb" }));
app.use(cors());

function requireKey(req, res, next) {
  if (req.get("x-api-key") !== API_KEY) return res.status(401).json({ error: "unauthorized" });
  next();
}

// ESP32 posts noise + radar here
app.post("/api/data", requireKey, async (req, res) => {
  const noise = Number(req.body.noise), radar = Number(req.body.radar);
  if (!Number.isFinite(noise) || (radar !== 0 && radar !== 1))
    return res.status(400).json({ error: "noise (number) and radar (0/1) required" });
  await Reading.create({ noise, radar });
  res.status(201).json({ ok: true });
});

// camera / people-counting program posts the head-count here
app.post("/api/occupancy", requireKey, async (req, res) => {
  const count = Number(req.body.count);
  if (!Number.isInteger(count) || count < 0 || count > 500)
    return res.status(400).json({ error: "count must be an integer 0-500" });
  await Occupancy.create({ count });
  res.status(201).json({ ok: true });
});

// dashboard reads the newest values here
app.get("/api/latest", async (req, res) => {
  const [r, o] = await Promise.all([
    Reading.findOne().sort({ createdAt: -1 }).lean(),
    Occupancy.findOne().sort({ createdAt: -1 }).lean(),
  ]);
  const now = Date.now();
  const camOn = !!o && now - o.createdAt.getTime() < CAM_ONLINE;
  res.set("Cache-Control", "no-store");
  res.json({
    esp32: r && now - r.createdAt.getTime() < ESP_ONLINE ? 1 : 0,
    noise: r ? r.noise : null,   radar: r ? r.radar : null,
    camera: camOn ? 1 : 0,       occupancy: camOn ? o.count : null,
  });
});

mongoose.connect(process.env.MONGODB_URI)
  .then(() => app.listen(PORT, () => console.log("Server on port " + PORT)))
  .catch((e) => { console.error(e.message); process.exit(1); });
```

SAMPLE SOURCE CODE

C. Dashboard (sample)

```
<!-- index.html (sample) -->
<h1>Smart Classroom Monitor</h1>
<div id="occ">--</div>
<div id="noise">--</div>
<div id="radar">--</div>
<div id="alerts">No active alerts</div>
<script src="script.js"></script>

// script.js (sample): polls the backend every second and shows status + alerts
const API_BASE   = "https://YOUR-BACKEND.onrender.com";
const NOISE_HIGH = 70;      // dB: above this the noise is Excessive
const NOISE_LOW  = 50;      // dB: below this the room is Quiet
const CAPACITY   = 40;      // set to the classroom capacity

const $ = (id) => document.getElementById(id);

function noiseClass(db) {
  if (db < NOISE_LOW) return "Quiet";
  if (db <= NOISE_HIGH) return "Normal";
  return "Excessive";
}

function occupancyClass(count) {
  const rho = count / CAPACITY;                 // occupancy ratio
  if (count === 0) return "Empty";
  if (rho < 0.8) return "Partly occupied";
  if (rho <= 1.0) return "Near capacity";
  return "Over capacity";
}

function render(d) {
  const alerts = [];
  if (d.occupancy === null) {
    $("occ").textContent = "Waiting for camera count";
  } else {
    $("occ").textContent = d.occupancy + " / " + CAPACITY + " - " + occupancyClass(d.occupancy);
    if (d.occupancy > CAPACITY) alerts.push("OCCUPANCY ALERT");
  }
  $("noise").textContent = d.noise.toFixed(1) + " dB - " + noiseClass(d.noise);
  if (d.noise > NOISE_HIGH) alerts.push("HIGH NOISE DETECTED");
  $("radar").textContent = d.radar === 1 ? "PRESENT" : "CLEAR";
  $("alerts").textContent = alerts.length ? alerts.join(" | ") : "No active alerts";
}

async function poll() {
  try {
    const res = await fetch(API_BASE + "/api/latest", { cache: "no-store" });
    const d = await res.json();
    if (d.esp32 === 1) render(d);
    else $("alerts").textContent = "No data from ESP32 in the last 10 seconds";
  } catch (err) {
    $("alerts").textContent = "Backend not reachable";
  } finally {
    setTimeout(poll, 1000);
  }
}
poll();
```

5.4 SAMPLE INPUT AND OUTPUT

5.4.1 Sample input

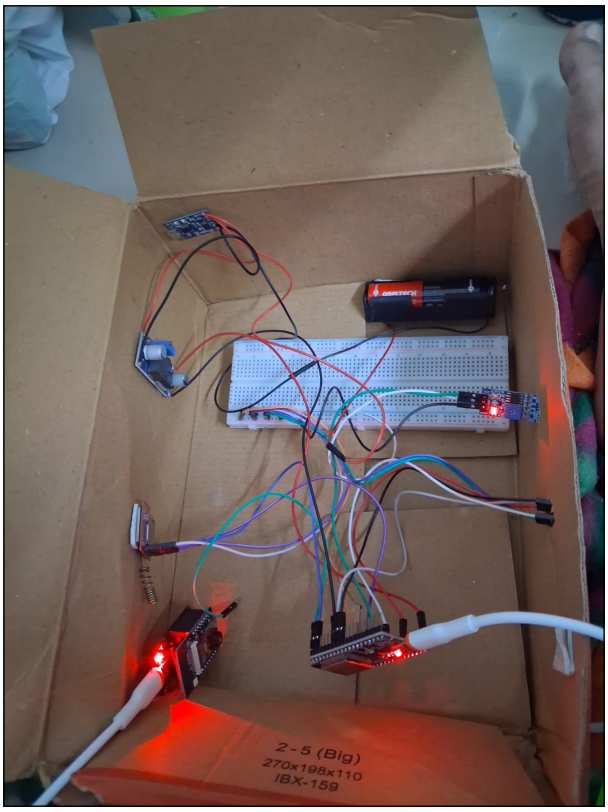


Fig. 5.3 Hardware setup of the prototype (sample input)

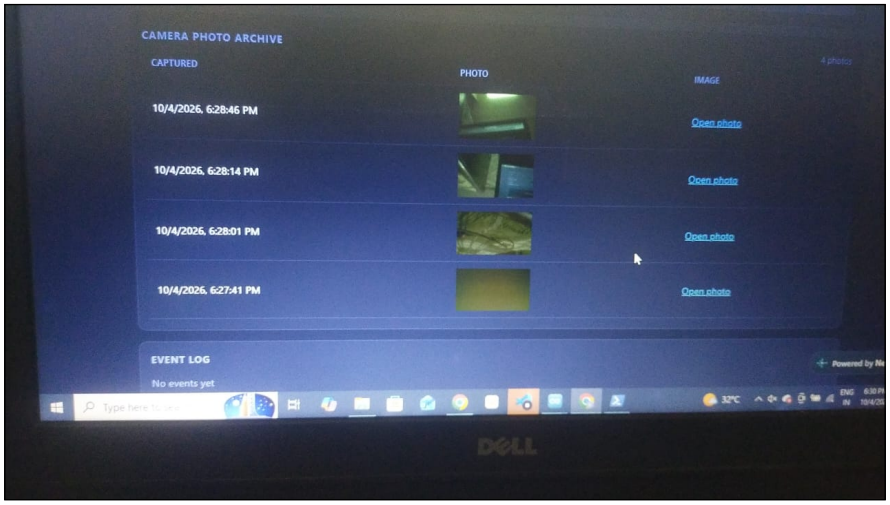


Fig. 5.4 Camera photo archive captured by the ESP32-CAM (sample input)

5.4.2 Sample output

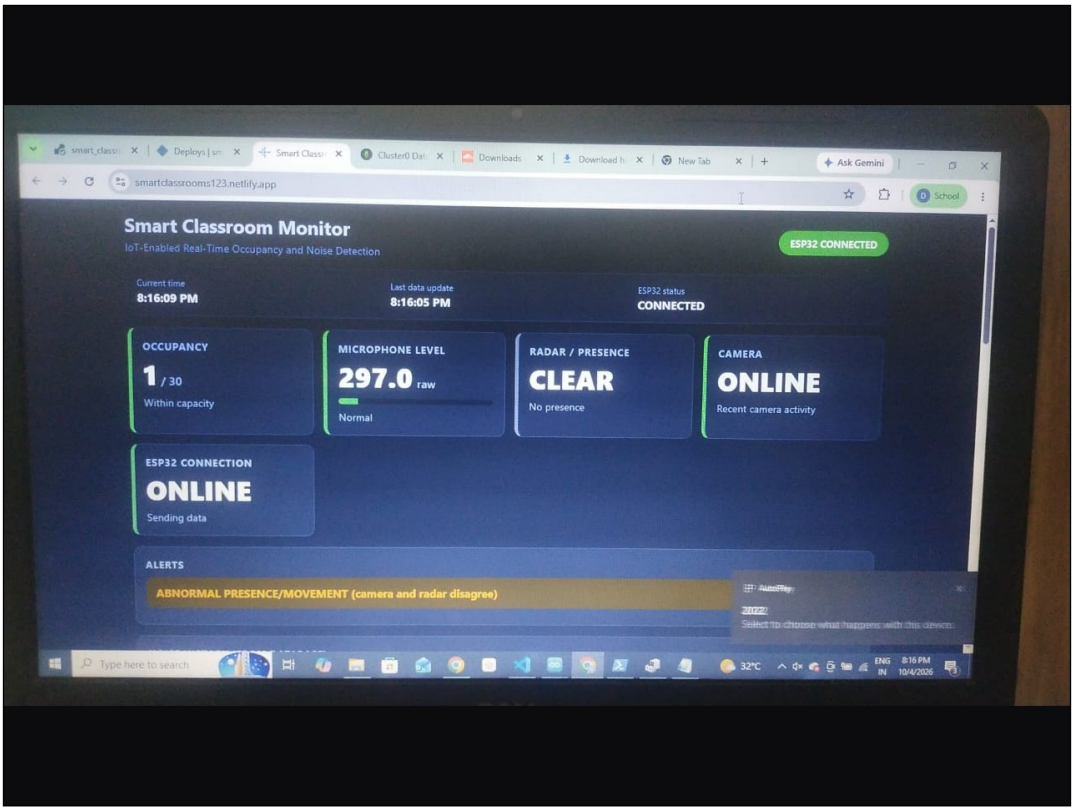


Fig. 5.5 Live dashboard showing microphone, radar and ESP32 status (sample output)

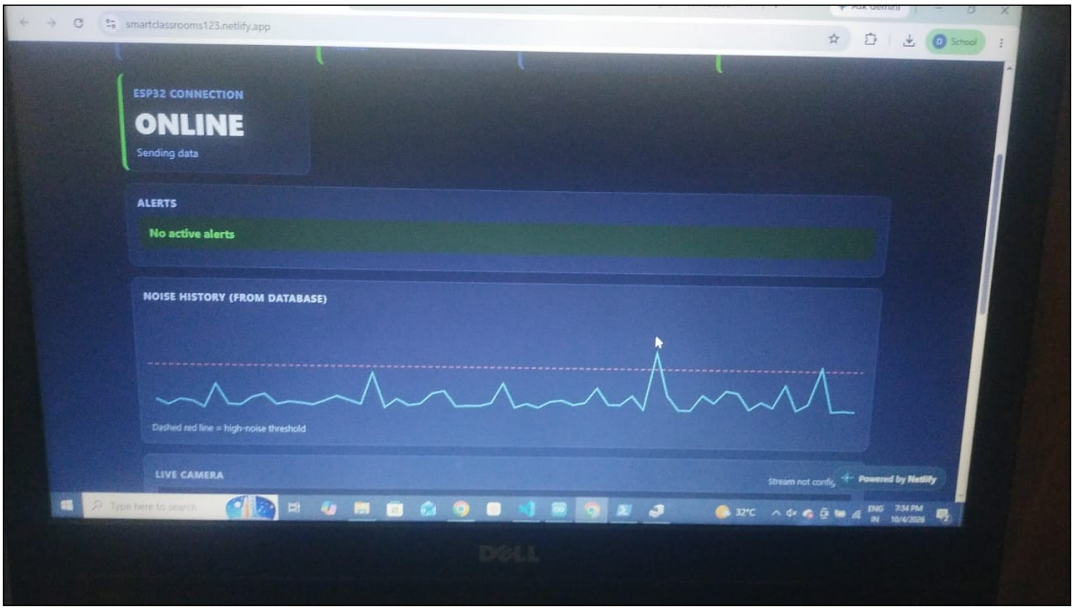


Fig. 5.6 Noise history graph with high-noise threshold (sample output)

REFERENCES

[1] ESP32-CAM and CNN Face Recognition for Classroom Attendance, International Conference ICCCES, 2026.

[2] Multi-Modal IoT and AI Based Faculty Presence Detection, International Journal of Research Studies in Science and Innovation (IJRSI), 2025.

[3] ESP32 Wi-Fi CAM Surveillance Robot, Conference IATMSI, 2025.

[4] Real-Time Noise Detection and Alert System, ResearchGate publication, 2025.

[5] AI and IoT-Enabled Smart Classroom Monitoring, International Journal of Engineering and Technology (IJET), 2025.

[6] Espressif Systems, ESP32 Series Datasheet.

[7] OmniVision, OV2640 Color CMOS UXGA CameraChip Datasheet.

[8] Hi-Link, LD2410 24 GHz Human Presence Sensing Radar Module Datasheet.



SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

PLATINUM

CERTIFICATE OF PARTICIPATION

This certificate is presented to

PRATHAMESH .T CHOKALE

Vels University.

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

An IoT Enabled Intelligent Classmonitoring Framework for Real-Time Occupancy and Noise Detection.

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
Dr. B. Ramesh



SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

PLATINUM

CERTIFICATE OF PARTICIPATION

This certificate is presented to

JONATHAN MERCIANO WHEELER

Vels University.

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

An IoT Enabled Intelligent classmonitoring Framework for Real-Time Occupancy and Noise Detection.

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
Dr. B. Ramesh



SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

PLATINUM

CERTIFICATE OF PARTICIPATION

This certificate is presented to

BHARATH SASIKUMAR

Vels University.

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

An IoT Enabled Intelligent Classmonitoring Framework for Real-Time Occupancy and Noise Detection.

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
Dr. B. Ramesh